

Refine ▼▼

Search for articles...

[Home](#) » [Wowza Streaming Engine](#) » [Supported streaming protoc...](#) » [WebRTC](#)

Ingest RTSP, SRT, or RTMP streams into Wowza Streaming Engine for playback with WebRTC

Updated on 04/20/2023 12:25 pm PDT

Learn how to use Wowza Streaming Engine™ media server software to ingest a non-WebRTC source stream over RTSP, SRT, or RTMP and play it back with WebRTC on mobile and desktop browsers that support WebRTC APIs.

This workflow provides flexibility with publishing streams to Wowza Streaming Engine while maintaining the benefits of WebRTC for playback, including:

- Low latency when compared with HTTP-based streaming.
- Browser-based playback without plugins or protocol-specific, proprietary players.
- Robust feature set of the open-source WebRTC framework.

In addition to outputting the stream for playback with WebRTC, you can also output the stream using any other playback protocols that Wowza Streaming Engine supports. This allows a scaled approach for delivery using a content delivery network (CDN) in addition to direct WebRTC browser connections.

Note: Wowza Streaming Engine™ media server software version 4.7.7 and later supports WebRTC streaming. However, we recommend that you update to version 4.8.5 or later to capitalize on expanded functionality and enhancements to publisher reliability.

Contents

Before you start

1. Configure a non-WebRTC to WebRTC workflow

Set up an RTSP source for a passthrough video-only stream

Set up an SRT source for a transcoded audio, passthrough video stream

Set up an RTMP source for a transcoded video and audio stream

2. (Optional) Tune for latency

3. Test WebRTC playback

Next steps

More resources

Before you start

Before you start, set up WebRTC playback according to [Set up WebRTC streaming with Wowza Streaming Engine](#).

1. Configure a non-WebRTC to WebRTC workflow

When planning a non-WebRTC to WebRTC workflow, consider whether you'll need to transcode the stream to achieve your goal of WebRTC playback.

By default, Wowza Streaming Engine transmuxes the stream into the HLS, MPEG-DASH, RTSP/RTP, and RTMP protocols for playback at scale. Transcoding is required when the ingest source stream has a different audio codec, video codec, or video encoding profile from the WebRTC output.

The workflows in this article provide a few possibilities for non-WebRTC ingest to WebRTC playback, but they aren't an exhaustive list. Transcoding introduces latency into the media delivery pipeline, so transcoded workflows will have a longer startup time and higher latency than a passthrough video-only workflow.

Set up an RTSP source for a passthrough video-only stream

In this example workflow, you ingest a stream from an IP camera or RTSP encoder into Wowza Streaming Engine, pass through the video and audio, and output a video-only WebRTC stream.

- **RTSP source** – H.264 video, AAC audio
- **WebRTC output** – H.264 video only

Note: If you require audio playback, you need to transcode the audio stream from the AAC audio codec to the Opus audio codec for WebRTC audio output.

See the steps 

Set up an SRT source for a transcoded audio, passthrough video stream

In this example workflow, you ingest an SRT stream from an encoder into Wowza Streaming Engine, transcode the audio codec from AAC to Opus, pass through the video, and output a WebRTC stream with video and audio.

- **SRT source** – H.264 video, AAC audio
- **WebRTC output** – H.264 video, Opus audio

See the steps 

Set up an RTMP source for a transcoded video and audio stream

In this example workflow, you'll ingest an RTMP stream from a camera or encoder into Wowza Streaming Engine, transcode the audio codec from AAC to Opus, transcode the video profile from High to Main, and output a WebRTC stream with video and audio.

- **RTMP source** – H.264 video with the High profile, AAC audio
- **WebRTC output** – H.264 video with the Main profile, Opus audio

See the steps 

2. (Optional) Tune for latency

The expected latency of passthrough streams that output WebRTC is one second or less. Transcoding, however, introduces latency into the media delivery pipeline. Streams that require audio or video transcoding have an expected latency of two seconds or less. While some amount of latency is expected due to frame buffering in your encoder, there are steps you can take to tune the WebRTC stream for low latency, depending on the source and whether it needs to be transcoded.

See the steps 

3. Test WebRTC playback

Now that your custom live application and source stream ingest is set up, you can test out your workflow. In production environments, a WebRTC playback page must be hosted on a web server utilizing SSL/TLS encryption. For testing and learning purposes, Wowza provides a hosted WebRTC playback test page so you can see WebRTC in action more quickly.

WebRTC test page requirements

- **Wowza Streaming Engine** – version 4.8.5 or later
- **Browsers** – latest version of Chrome, Firefox, Safari, and Microsoft Edge version 79 and later. There is a resolved known issue with publishing and playing WebRTC streams using Safari on iOS 15. See WebRTC publishing and playback fails with Safari on iOS 15 for updates.
- **Screen share** – not supported on mobile devices, Safari, or Firefox

1. Ensure that your IP camera or encoder is sending a source stream to Wowza Streaming Engine.
2. In a browser tab, go to the WebRTC play hosted test page.
3. In the **Signaling URL** field, enter the secure WebSocket URL to connect to the Wowza Streaming Engine WebRTC sessions listener:

`wss://[ssl-certificate-domain-name]/webrtc-session.json`

where *ssl-certificate-domain-name* is the secure domain name for your Wowza Streaming Engine instance.

If using Wowza StreamLock, for example, the **Signaling URL** looks something like this:

`wss://5ab4321c0d123.streamlock.net/webrtc-session.json`

If you are connecting WebRTC sessions using a port other than the standard SSL/TLS port 443, you must include that non-standard port in the **Signaling URL**:

`wss://5ab4321c0d123.streamlock.net:[SSL-port-number]/webrtc-session.json`

4. Enter an **Application Name** that matches the WebRTC live application you configured.
5. For **Stream Name**, enter a name for the stream, such as **myStream**.
6. To play the WebRTC stream from Wowza Streaming Engine, click **Play**.

To test playback in a different browser or with a different device, click **Copy config** () to copy the configuration settings and share them.

Tip: When you're ready to implement WebRTC for production, consider using the Wowza Flowplayer Real-Time Streaming (WebRTC) plugin for playback. You can purchase Wowza Flowplayer and use it with Wowza Streaming Engine for playback.

Note: To test playback over other protocols, enable other playback protocols for the application you configured for WebRTC by clicking the **Setup** tab for your application and enabling any of the protocols listed under **Playback Types**. Then, you can use the playback URL for the protocol you want to test with the player of your choice or the Video Test Players webpage.

Next steps

- For more advanced learning and testing, Wowza Media Systems provides WebRTC examples on GitHub that demonstrate how to play WebRTC streams with Wowza Streaming Engine. See Use WebRTC example pages with Wowza Streaming Engine.

More resources

- WebRTC workflows in Wowza Streaming Engine
- Control access to WebRTC publishing and playback
- Record WebRTC streams with Wowza Streaming Engine